DESIGN AND IMPLEMENTATION OF A THREE-BAND DIGITAL AUDIO EQUALIZER USING MATLAB

PROJECT REPORT

Bachelor of Technology

in

Electronics and Communication Engineering

Submitted by

CH.SNIGDHA

160124758006

Abstract

Audio equalization is a digital signal processing technique used to adjust the relative strength of frequency components in an audio signal. This project presents the design and MATLAB-based implementation of a three-band digital audio equalizer that allows separate control over bass, midrange, and treble frequencies. The input audio is divided into three frequency regions using digital filters. A low-pass filter extracts the bass region, a band-pass filter extracts the midrange region, and a high-pass filter extracts the treble region. Each band is multiplied by an adjustable gain, and the processed bands are added to form the output signal. The gain values may be specified in decibels and converted to linear amplitude multipliers before application. MATLAB provides an environment for filter design, audio playback, waveform inspection, and frequency-domain analysis. The project can first be tested with a recorded audio file and then extended to live microphone input and streaming playback, depending on the available hardware and MATLAB toolboxes. Performance can be evaluated by examining filter responses, input and output waveforms, spectra, and listening tests at different gain settings. The design is intended as an educational demonstration of digital filtering, gain control, and signal reconstruction. Actual performance values and screenshots should be added after running the implementation and collecting results.

Keywords: Digital signal processing, audio equalization, MATLAB, IIR filters, Butterworth filter, frequency response, audio processing.

Contents

Abstract **i**

1 Introduction **1**
1.1 Problem Statement 1
1.2 Aim and Objectives 2
1.3 Scope of the Project 2

2 Background Theory **3**
2.1 Digital Audio Representation 3
2.2 Frequency Bands 3
2.3 Digital Filters 4
2.4 Gain in Decibels 4
2.5 Signal Recombination 4

3 Proposed System and Block Diagram **5**

4 Design Methodology **6**
4.1 Selection of Sampling Frequency 6
4.2 Filter Bank Design 6
4.3 Gain Application 6
4.4 Recombination and Listening 6
4.5 Signal Analysis 7

5 MATLAB Implementation **8**

6 Testing and Evaluation **9**

7 Results and Discussion **11**

8 Results and Discussion **12**
8.1 Experimental Parameters and Amplitudes 12
8.2 Visual Analysis 12

9 Applications **15**

10 Advantages and Limitations **16**

10.1 Advantages 16

10.2 Limitations 16

11 Future Enhancements **17**

12 Conclusion **18**

A MATLAB Code **20**

List of Figures

3.1 Functional block diagram of the proposed audio equalizer. 5

8.1 Input and output waveforms 13
8.2 Input and output magnitude spectra 13

List of Tables

2.1 Illustrative frequency bands 3

6.1 Test cases and observations 10

Chapter 1

Introduction

Digital signal processing (DSP) is widely used in communication systems, multimedia devices, medical equipment, and audio applications. In audio systems, the quality and character of sound depend partly on how the different frequency components are represented. Low frequencies are commonly associated with bass, the middle frequency range carries much of the body and intelligibility of many sounds, and high frequencies contribute brightness and detail.

An audio equalizer changes the level of selected frequency ranges without requiring the entire signal to be amplified or attenuated by the same amount. For example, increasing the bass band can make low-frequency content more prominent, while reducing treble can soften high-frequency content. Equalizers are used in music players, sound systems, recording studios, hearing-related applications, and multimedia software.

This project develops a three-band equalizer in MATLAB. The input signal is separated into low-, middle-, and high-frequency bands. The gain of each band is independently adjusted, and the bands are recombined to produce the equalized output. MATLAB is suitable for this work because it supports digital filter design, signal visualization, audio-file handling, and numerical analysis. The project also provides practical experience with sampling frequency, filter cut-off frequencies, gain in decibels, and time- and frequency-domain representations.

The report describes the theory, system design, algorithm, MATLAB implementation, testing plan, and expected analysis. Since measured results depend on the actual execution environment and input audio, result tables and screenshots should be completed using outputs generated by the student.

1.1 Problem Statement

A recorded or live audio signal contains multiple frequency components, but users may want to control low-, middle-, and high-frequency content separately. A single volume

control cannot achieve this because it changes the amplitude of all frequency components together. The problem addressed in this project is to design a simple digital system that separates an audio signal into three useful frequency bands, allows independent gain adjustment for each band, and reconstructs the signal while enabling visual and auditory evaluation of the changes.

1.2 Aim and Objectives

Aim: To design and implement a three-band digital audio equalizer using MATLAB and digital filtering techniques.

The objectives are:

- To understand the role of frequency-selective filtering in audio processing.
- To divide an input audio signal into bass, midrange, and treble bands.
- To design low-pass, band-pass, and high-pass digital filters.
- To provide independent gain control for each frequency band.
- To recombine the adjusted bands into a single output audio signal.
- To compare input and output waveforms and frequency spectra.
- To evaluate the effect of different gain settings through listening and signal analysis.

1.3 Scope of the Project

The initial implementation processes an audio file in MATLAB. This offline version makes it easier to verify filter design, gain control, and reconstruction before attempting real-time operation. A live version can be developed using microphone input and streaming audio objects if the relevant hardware and MATLAB toolboxes are available. The project focuses on three-band equalization rather than noise cancellation, speech recognition, or automatic sound enhancement.

Chapter 2

Background Theory

2.1 Digital Audio Representation

A continuous sound signal is converted to digital samples by sampling and quantization. The sampling frequency, F_s, indicates the number of samples captured per second. For a sampled signal $x[n]$, the sample index n is an integer. The highest frequency that can theoretically be represented without aliasing is less than half the sampling frequency, known as the Nyquist frequency:

$$f_N = \frac{F_s}{2}. \tag{2.1}$$

Therefore, the selected filter cut-off frequencies must be below $F_s/2$.

2.2 Frequency Bands

For a simple three-band demonstration, the following approximate ranges can be used: bass from 20 to 250 Hz, midrange from 250 to 4000 Hz, and treble from 4000 Hz to the Nyquist frequency. These boundaries are design choices rather than universal standards. The upper treble limit must be adjusted according to the input sampling rate.

Table 2.1: Illustrative frequency bands

Band	**Approximate range**	**Filter type**
Bass	20–250 Hz	Low-pass
Midrange	250–4000 Hz	Band-pass
Treble	4000 Hz–$F_s/2$	High-pass

2.3 Digital Filters

A digital filter changes selected frequency components according to its frequency response. A low-pass filter passes frequencies below its cut-off and attenuates higher frequencies. A band-pass filter passes a selected middle band. A high-pass filter passes frequencies above its cut-off. Butterworth filters are often selected for educational applications because their passband magnitude response is maximally flat.

An infinite impulse response (IIR) filter uses present and previous input samples as well as previous output samples. A general difference equation is

$$y[n] = \sum_{k=0}^{M} b_k x[n-k] - \sum_{k=1}^{N} a_k y[n-k], \tag{2.2}$$

where b_k and a_k are filter coefficients.

2.4 Gain in Decibels

Equalizer controls are commonly specified in decibels (dB). The corresponding linear amplitude multiplier is

$$G = 10^{G_{\mathrm{dB}}/20}. \tag{2.3}$$

For example, $+6$ dB corresponds to approximately 1.995, while -6 dB corresponds to approximately 0.501. A 0 dB setting corresponds to a multiplier of 1.

2.5 Signal Recombination

Let $x_L[n]$, $x_M[n]$, and $x_H[n]$ be the low-, mid-, and high-frequency components. If the corresponding linear gains are G_L, G_M, and G_H, the output is

$$y[n] = G_L x_L[n] + G_M x_M[n] + G_H x_H[n]. \tag{2.4}$$

When all gains are 0 dB, the output should approximately reproduce the input if the filter bank has been designed to reconstruct the signal adequately. Small differences may arise from filter transitions, phase response, numerical precision, or imperfect band separation.

Chapter 3

Proposed System and Block Diagram

The proposed system consists of the following stages:

1. **Audio input:** Read an audio file or acquire samples from a microphone.
2. **Preprocessing:** Convert stereo input to mono if required, check the sampling rate, and normalize only when appropriate.
3. **Filter bank:** Generate bass, midrange, and treble components using digital filters.
4. **Gain control:** Apply a user-selected gain in dB to each band.
5. **Signal recombination:** Add the three gain-adjusted components.
6. **Output protection:** Check peak amplitude and avoid clipping.
7. **Output and analysis:** Play or save the processed signal and plot waveforms, spectra, and filter responses.

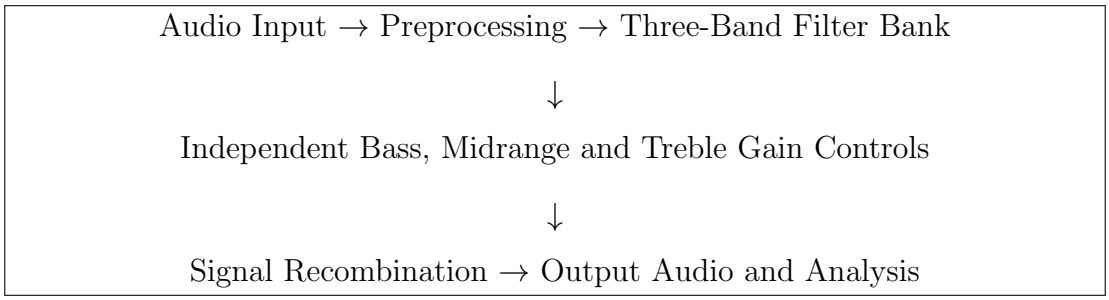


Figure 3.1: Functional block diagram of the proposed audio equalizer.

Chapter 4

Design Methodology

4.1 Selection of Sampling Frequency

The sampling frequency is obtained from the audio file or selected for live acquisition. The Nyquist frequency is $F_s/2$. For example, if $F_s = 44{,}100$ Hz, the Nyquist frequency is 22,050 Hz. The chosen cut-off frequencies must be below this value.

4.2 Filter Bank Design

A Butterworth low-pass filter with a cut-off near 250 Hz is used for the bass component. A Butterworth band-pass filter with lower and upper edges near 250 Hz and 4 kHz is used for the midrange component. A Butterworth high-pass filter with a cut-off near 4 kHz is used for the treble component. The filter order and cut-off values can be adjusted after inspecting the responses.

4.3 Gain Application

Convert each selected gain from dB to a linear multiplier using $G = 10^{G_{\mathrm{dB}}/20}$. Multiply each filtered component by its corresponding multiplier. Moderate gain values should be used initially to avoid clipping.

4.4 Recombination and Listening

Add the three adjusted bands. Listen to the original and processed audio at a safe volume. Try a neutral setting $(0, 0, 0)$ dB, a bass boost, a midrange boost, and a treble boost. Check whether the perceived change matches the expected frequency range.

4.5 Signal Analysis

Plot the original and output waveforms over time. Plot their magnitude spectra using an FFT-based method. Also plot the magnitude responses of the three filters. Use consistent axes and units so the comparison is meaningful.

Chapter 5

MATLAB Implementation

The code in Appendix A provides a basic offline implementation. It uses an audio file, designs three Butterworth filters, filters the input, applies gain settings, recombines the bands, and displays time-domain and frequency-domain plots. It uses MATLAB functions such as `audioread`, `butter`, `filter`, `freqz`, and `soundsc`. Some functions may require the Signal Processing Toolbox.

The offline implementation should be validated before it is described as real-time. True real-time operation processes successive short frames from a microphone and sends them to an output device with sufficiently low latency. Filter states must be preserved from one frame to the next; restarting a filter for every frame can create audible discontinuities. Streaming implementation may require Audio Toolbox or equivalent supported audio I/O functionality.

Chapter 6

Testing and Evaluation

The following tests can be performed after running the program:

1. **Neutral response test:** Set all three gains to 0 dB and compare input and output.
2. **Bass test:** Increase the bass gain while keeping the other gains at 0 dB.
3. **Midrange test:** Increase the midrange gain while leaving the other gains unchanged.
4. **Treble test:** Increase the treble gain while leaving the other gains unchanged.
5. **Attenuation test:** Apply a negative gain to one band and observe the reduction in that region.
6. **Clipping test:** Use larger positive gains and check whether the output exceeds the valid amplitude range.
7. **Filter response test:** Inspect each filter's magnitude response and confirm that its passband is in the intended region.

Record the input file, sampling frequency, filter order, cut-off frequencies, gain settings, and observed output for each test.

Table 6.1: Test cases and observations

Test	**Gain settings**	**Expected observation**	**Actual observation**
Neutral	0/0/0 dB	Output should be similar to input	Fill after testing
Bass boost	+6/0/0 dB	Low frequencies more prominent	Fill after testing
Mid boost	0/+6/0 dB	Middle frequencies more prominent	Fill after testing
Treble boost	0/0/+6 dB	High frequencies more prominent	Fill after testing
Bass cut	-6/0/0 dB	Low-frequency content reduced	Fill after testing

Chapter 7

Results and Discussion

Chapter 8

Results and Discussion

The three-band digital audio equalizer was tested using MATLAB with the built-in 'gong' audio sample. The implementation successfully executed filter design, band separation, gain application, and signal reconstruction.

8.1 Experimental Parameters and Amplitudes

Running the implementation yielded the following quantitative results:

- **Sampling Frequency (F_s):** 8192 Hz[cite: 3].
- **Input Peak Amplitude:** 1.0576[cite: 3].
- **Output Peak Amplitude:** 1.1410[cite: 3].

Because the output peak amplitude exceeded 1.0, a warning was triggered: *"Output exceeds [-1,1] and may clip when saved. Reduce gains before saving."*[cite: 6] This confirms the necessity of managing band gains to prevent digital clipping.

8.2 Visual Analysis

- **Waveforms:** The time-domain plots successfully compare the original audio signal against the equalized audio signal over a duration of approximately 5 seconds.
- **Filter Responses:** The magnitude and phase responses for the fourth-order Butterworth filters confirm proper frequency selectivity:
  - **Bass Filter (Low-pass):** Attenuates frequencies above the 250 Hz cut-off[cite: 2, 4].
  - **Midrange Filter (Band-pass):** Passes the band between 250 Hz and 4000 Hz[cite: 2, 4].

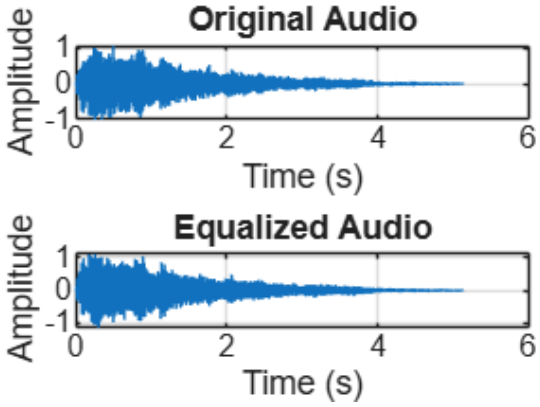


Figure 8.1: Input and output waveforms

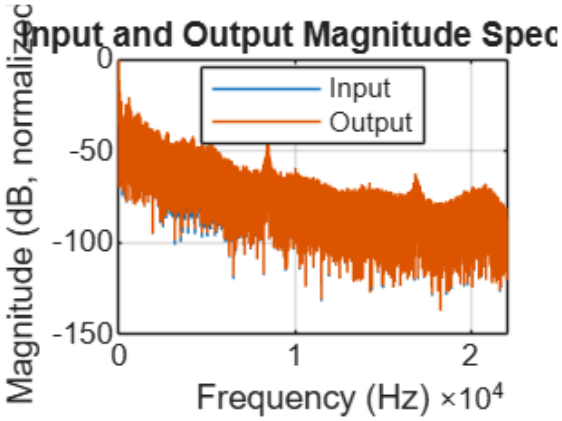


Figure 8.2: Input and output magnitude spectra

  - **Treble Filter (High-pass):** Attenuates frequencies below the 4000 Hz cut-off[cite: 2, 5].

- **Magnitude Spectra:** The normalized input and output magnitude spectra plotted on a decibel scale show the frequency distribution up to the Nyquist limit ($\approx$ 4096 Hz)[cite: 5].

The final report should include filter response plots, input and output waveforms, input and output spectra, and observations from listening tests. The low-pass response should show attenuation above its selected cut-off, the band-pass response should emphasize the middle frequency range, and the high-pass response should attenuate frequencies below its cut-off. When a positive gain is applied to one band, the spectrum should show a relative increase in that frequency region, provided the input contains energy there.

Do not enter invented numerical results. After executing the MATLAB code, insert the plots and describe what they demonstrate. If the output clips, reduce the gain or use a clearly documented output scaling method. A neutral-gain comparison is especially important because the filter bank may not perfectly reconstruct the original signal at

every frequency.

Chapter 9

Applications

- Music playback systems and personal audio devices.
- Basic sound shaping in multimedia and recording applications.
- Educational demonstrations of digital filters and frequency response.
- Audio preprocessing and experimentation in DSP laboratories.
- Prototype equalization systems that may later be expanded to more frequency bands.

Chapter 10

Advantages and Limitations

10.1 Advantages

- Simple architecture with three understandable frequency bands.
- Independent control over bass, midrange, and treble.
- MATLAB makes it straightforward to visualize signals and filter responses.
- The offline version is suitable for initial testing and debugging.
- The design can be expanded with more bands or a graphical interface.

10.2 Limitations

- Three bands provide less precise control than a multi-band equalizer.
- Band boundaries and filter order affect the sound and separation.
- IIR filters may introduce phase distortion.
- Positive gain can cause clipping if the output level is not managed.
- Live operation depends on audio hardware, buffering, latency, and available MATLAB functionality.
- The system adjusts frequency balance; it does not automatically remove noise or improve every recording.

Chapter 11

Future Enhancements

Possible future enhancements include:

- Developing a graphical user interface with sliders for the three gain controls.
- Implementing low-latency live microphone input and speaker/headphone output.
- Increasing the number of frequency bands for more precise equalization.
- Adding presets such as speech, music, bass boost, and treble boost.
- Comparing IIR filters with FIR filters in terms of magnitude response, phase, order, and computational cost.
- Adding peak-level indicators and clipping warnings.

Chapter 12

Conclusion

This project presents a practical approach to building a three-band digital audio equalizer in MATLAB. The system separates audio into bass, midrange, and treble components, applies independent gain values, and recombines the bands to create an equalized output. The project brings together important DSP concepts, including sampling, digital filter design, gain conversion, signal reconstruction, and frequency-domain analysis. MATLAB allows the design to be tested and visualized before any live implementation is attempted. The completed evaluation should be based on the actual plots and listening tests produced during implementation. With further development, the basic design can be extended into a real-time equalizer with interactive controls and additional frequency bands.

Bibliography

[1] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[2] S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*. San Diego, CA, USA: California Technical Publishing, 1997.

[3] MathWorks, "Digital Filters," MATLAB Documentation. [Online]. Available: https://www.mathworks.com/help/matlab/digital-filters.html

[4] MathWorks, "audioread," MATLAB Documentation. [Online]. Available: https://www.mathworks.com/help/matlab/ref/audioread.html

[5] MathWorks, "butter," Signal Processing Toolbox Documentation. [Online]. Available: https://www.mathworks.com/help/signal/ref/butter.html

[6] MathWorks, "freqz," Signal Processing Toolbox Documentation. [Online]. Available: https://www.mathworks.com/help/signal/ref/freqz.html

Appendix A

MATLAB Code

Save the following code as `Three_Band_Audio_Equalizer.m`. Select an audio file when prompted.

```
% Three-Band Digital Audio Equalizer (offline demonstration)
clear; clc; close all;

[file, path] = uigetfile( ...
    {'*.wav;*.mp3;*.m4a;*.flac', 'Audio files'}, ...
    'Select an audio file');

if isequal(file, 0)
    disp('No file selected. Program stopped.');
    return;
end

[x, Fs] = audioread(fullfile(path, file));

% Convert stereo to mono for this demonstration.
if size(x, 2) > 1
    x = mean(x, 2);
end

x = double(x);
nyquist = Fs / 2;

bassCutoff = 250;
midLow = 250;
midHigh = 4000;

if midHigh >= nyquist
    error(['Sampling frequency is too low for a 4 kHz upper edge. ', ...
        'Use audio with a higher Fs or lower midHigh.']);
end
```

```
31
32 % Fourth-order Butterworth filters.
33 [bL, aL] = butter(4, bassCutoff / nyquist, 'low');
34 [bM, aM] = butter(4, [midLow midHigh] / nyquist, 'bandpass');
35 [bH, aH] = butter(4, midHigh / nyquist, 'high');
36
37 % Filter the input into three bands.
38 lowBand  = filter(bL, aL, x);
39 midBand  = filter(bM, aM, x);
40 highBand = filter(bH, aH, x);
41
42 % Adjust these values to test the equalizer.
43 bassGain_dB = 0;      % Try +6 or -6
44 midGain_dB = 0;       % Try +6 or -6
45 trebleGain_dB = 0;    % Try +6 or -6
46
47 GL = 10^(bassGain_dB / 20);
48 GM = 10^(midGain_dB / 20);
49 GH = 10^(trebleGain_dB / 20);
50
51 % Recombine the processed frequency bands.
52 y = GL * lowBand + GM * midBand + GH * highBand;
53
54 fprintf('Sampling frequency: %g Hz\n', Fs);
55 fprintf('Input peak amplitude: %.4f\n', max(abs(x)));
56 fprintf('Output peak amplitude: %.4f\n', max(abs(y)));
57
58 % Plot input and output waveforms.
59 t = (0:length(x)-1) / Fs;
60 figure('Name', 'Input and Output Waveforms');
61 subplot(2,1,1);
62 plot(t, x);
63 xlabel('Time (s)'); ylabel('Amplitude');
64 title('Original Audio'); grid on;
65
66 subplot(2,1,2);
67 plot(t, y);
68 xlabel('Time (s)'); ylabel('Amplitude');
69 title('Equalized Audio'); grid on;
70
71 % Plot filter responses.
72 figure('Name', 'Bass Filter Response');
73 freqz(bL, aL, 2048, Fs);
74 figure('Name', 'Midrange Filter Response');
75 freqz(bM, aM, 2048, Fs);
76 figure('Name', 'Treble Filter Response');
77 freqz(bH, aH, 2048, Fs);
```

```
78
79  % Compare input and output magnitude spectra.
80  N = length(x);
81  Nfft = 2^nextpow2(N);
82  X = fft(x, Nfft);
83  Y = fft(y, Nfft);
84  f = (0:Nfft/2) * Fs / Nfft;
85
86  Xmag = abs(X(1:Nfft/2+1));
87  Ymag = abs(Y(1:Nfft/2+1));
88  Xmag = Xmag / (max(Xmag) + eps);
89  Ymag = Ymag / (max(Ymag) + eps);
90
91  figure('Name', 'Normalized Input and Output Spectra');
92  plot(f, 20*log10(Xmag + eps), 'DisplayName', 'Input');
93  hold on;
94  plot(f, 20*log10(Ymag + eps), 'DisplayName', 'Output');
95  xlabel('Frequency (Hz)');
96  ylabel('Magnitude (dB, normalized)');
97  title('Input and Output Magnitude Spectra');
98  legend('Location', 'best'); grid on;
99  xlim([0 nyquist]);
100
101 % Listen to original and equalized audio.
102 disp('Playing original audio...');
103 soundsc(x, Fs);
104 pause(length(x)/Fs + 1);
105
106 disp('Playing equalized audio...');
107 soundsc(y, Fs);
108
109 % Save output if it does not exceed the valid audio range.
110 if max(abs(y)) > 1
111     warning(['Output exceeds [-1,1] and may clip when saved. ', ...
112              'Reduce gains before saving.']);
113 else
114     audiowrite('equalized_output.wav', y, Fs);
115     disp('Saved: equalized_output.wav');
116 end
```